Fix jit can not recognize the imported function #63248

Merged qihqi merged 5 commits into gh/qihan/1/base from gh/qihan/1/head on Sep 29, 2021

1 test/jit/_imported_class_test/__init__.py

```
@@ -0,0 +1 @@
+ from .foo import MyObj, inner, Nada, simple_func, NoConstructor
```

4 test/jit/_imported_class_test/bar.py

```
@@ -7,3 +7,7 @@
 class FooSameName(object):  # noqa: B903
     def __init__(self, y):
         self.y = y
+
+
+def f(x):
+    return x
```

38 test/jit/_imported_class_test/foo.py

```
@@ -1,5 +1,6 @@
import torch
from . import bar
+from .bar import f
# This file contains definitions of script classes.
# They are used by test_jit.py to test ScriptClass imports

@@ -9,3 +10,40 @@ class FooSameName(object):
    def __init__(self, x):
        self.x = x
        self.nested = bar.FooSameName(x)
+
+
+class MyObj(object):
+    def __init__(self):
+        pass
+
+    def __call__(self, x):
+        return f(x)
+
+
+def simple_func(x):
+    return f(x)
+
+
+class Nada:
+    pass
+
+
+class NoConstructor:
+
+    def g(self, x):
+        return f(x)
+
+
+def inner():
+
+    def xyz(x):
+        return x
```

```
41 +
42 +     class MyObj(object):
43 +         def __init__(self):
44 +             pass
45 +
46 +         def __call__(self, x):
47 +             return xyz(x)
48 +
49 +     return MyObj
```

10 test/jit/test_class_type.py

```
@@ -1538,3 +1538,13 @@ def fn_e():
1538 1538          for fn in (fn_a, fn_b, fn_c, fn_d, fn_e):
1539 1539              with self.assertRaisesRegex(RuntimeError, error_message_regex):
1540 1540                  torch.jit.script(fn)
1541 +
1542 +     @unittest.skipIf(IS_SANDCASTLE, "Importing like this doesn't work in fbcode")
1543 +     def test_jit_script_not_as_decorator(self):
1544 +         from jit._imported_class_test import MyObj, inner, Nada, simple_func, NoConstructor
1545 +         # Assert that the below does not raise
1546 +         torch.jit.script(MyObj)  # class that uses a symbol defined in another module
1547 +         torch.jit.script(inner())  # class that uses a symbol in closure
1548 +         torch.jit.script(Nada)  # empty class
1549 +         torch.jit.script(simple_func)  # regular function
1550 +         torch.jit.script(NoConstructor)  # class with no constructor
```

29 test/test_jit.py

```
@@ -336,6 +336,21 @@ def __init__(self):
336 336          super(FooToPickle, self).__init__()
337 337          self.bar = torch.jit.ScriptModule()
338 338
339 +
340 + class SomeClass(object):
341 +     def __init__(self):
342 +         self.num = 99
```

```
                343 +
                344 +     def __iadd__(self, x):
                345 +         # type: (int)
                346 +         self.num += x
                347 +         return self
                348 +
                349 +     def __eq__(self, other):
                350 +         # type: (SomeClass) -> bool
                351 +         return self.num == other.num
                352 +
                353 +
339             354   class TestJit(JitTestCase):
340             355       @unittest.skip("Requires a lot of RAM")
341             356       def test_big(self):
                      @@ -4087,19 +4102,7 @@ def fn():
4087            4102                  a += SomeNonAddableClass()
4088            4103                  return a
4089            4104
4090                    -         @torch.jit.script
4091                    -         class SomeClass(object):
4092                    -             def __init__(self):
4093                    -                 self.num = 99
4094                    -
4095                    -             def __iadd__(self, x):
4096                    -                 # type: (int)
4097                    -                 self.num += x
4098                    -                 return self
4099                    -
4100                    -             def __eq__(self, other):
4101                    -                 # type: (SomeClass) -> bool
4102                    -                 return self.num == other.num
                4105 +         torch.jit.script(SomeClass)
4103            4106
4104            4107          @torch.jit.script
4105            4108          class SomeOutOfPlaceClass(object):
```

41 torch/_jit_internal.py

```
@@ -160,15 +160,41 @@ def __getattr__(self, key):
160 160         return createResolutionCallbackFromEnv(env())
161 161
162 162
    163 + def get_closure_class(clazz):
    164 +     """
    165 +     Get a dictionary of closed over variables from a class.
    166 +
    167 +     The closure of a class is defined as the union of closure of it's member functions.
    168 +     """
    169 +     captures = {}
    170 +     captures[clazz.__name__] = clazz
    171 +     functions = inspect.getmembers(clazz, predicate=inspect.isfunction)
    172 +     for _, datadesc in inspect.getmembers(clazz, predicate=inspect.isdatadescriptor):
    173 +         # Case of getter / setter marked with @property / @something.setter
    174 +         # Fetch getter and setter
    175 +         functions.extend(inspect.getmembers(datadesc, predicate=inspect.isfunction))
    176 +     for _, member in functions:
    177 +         captures.update(get_type_hint_captures(member))
    178 +         captures.update(get_closure(member))
    179 +     return captures
    180 +
    181 +
163 182 def get_closure(fn):
164 183     """
165 184     Get a dictionary of closed over variables from a function
166 185     """
167 186     captures = {}
168 187     captures.update(fn.__globals__)
169     -
170 188     for index, captured_name in enumerate(fn.__code__.co_freevars):
171     -         captures[captured_name] = fn.__closure__[index].cell_contents
    189 +         try:
```



zhxchen17 on Aug 18, 2021 Contributor •••

when we have ValueError?

All reactions



qihqi on Aug 19, 2021 Contributor Author •••

This happens when torch.jit.script is used as decorator and the a method of class refers the class by name. Added comments.

All reactions



zhxchen17 on Aug 19, 2021 Contributor •••

I think it's better to use hasattr() to check existence of cell_contents, that way the intention is more clear and we can avoid accidentally capture unintended errors.

All reactions



qihqi on Sep 7, 2021 Contributor Author •••

hasattr returns True though... i.e. that field exists, it just throws when accessed before it's setup completely :(

All reactions

```
                190 +                captures[captured_name] = fn.__closure__[index].cell_contents
                191 +            except ValueError:
                192 +                # access to cell_contents raises if it's empty. The only time this
                193 +                # happens is when @torch.jit.script is used as decorator to a
                194 +                # class, and inside of some methods refers that class by name.
                195 +                # Python preallocates a cell and fill it with what the decorator returns,
                196 +                # so it's empty while torch.jit.script is running.
                197 +                pass
172             198
173             199         return captures
174             200
                    @@ -220,10 +246,15 @@ def get_closure(fn):
220             246
221             247     def createResolutionCallbackFromClosure(fn):
222             248         """
223             -       Create a resolutionCallback by introspecting the function instead of
                249 +       Create a resolutionCallback by introspecting the function or class instead of
224             250         looking up the stack for the enclosing scope
                251 +
```

```
                252 +     A closure of a class is defined as the union of closure of it's member
                    functions.
225             253       """
226             -     closure = get_closure(fn)
                254 +     if inspect.isclass(fn):
                255 +         closure = get_closure_class(fn)
                256 +     else:
                257 +         closure = get_closure(fn)
227             258
228             259       class closure_lookup(object):
229             260           # This is a class since `closure` is a dict and it's easier in
                    @@ -388,7 +419,7 @@ def createResolutionCallbackForClassMethods(cls):
388             419       # cls is a type here, so `ismethod` is false since the methods on the type
389             420       # aren't bound to anything, so Python treats them as regular functions
390             421       fns = [getattr(cls, name) for name in cls.__dict__ if
                    inspect.isroutine(getattr(cls, name))]
391             -     captures = {}
                422 +     captures = {cls.__name__: cls}
392             423
393             424       for fn in fns:
394             425           captures.update(get_closure(fn))
```

2 torch/jit/_script.py

```
                    @@ -1291,7 +1291,7 @@ def forward(self, a) -> MyModule:
1291            1291                    "Please directly inherit from 'object'."
1292            1292                )
1293            1293            if _rcb is None:
1294            -               _rcb = _jit_internal.createResolutionCallbackFromFrame(_frames_up + 1)
                1294 +         _rcb = _jit_internal.createResolutionCallbackFromClosure(obj)
```

zhxchen17 on Aug 18, 2021 — Contributor

I'm surprised that this actually works before, considering the difference between dynamic and lexical resolution.

Could we rename them to `createResolutionCallbackFromLexicalScope` and `createResolutionCallbackFromDynamicScope` to avoid some misuse in the future?

All reactions

qihqi on Aug 18, 2021 — Contributor, Author

hmmmm not really, createResolutionCallbackFromFrame doesn't give you the lexical scope of the class; but the lexical scope at the point torch.jit.script is called.

All reactions

zhxchen17 on Aug 19, 2021 — Contributor

Update: after an online discussion, we think it's worthwhile to investigate why createResolutionCallbackFromFrame() was there in the first place. If it doesn't make sense we can just remove it, so that the new logic will be much simpler. If it's there for a reason, then I'm not keen on the renaming thing.

All reactions

qihqi on Sep 7, 2021 — Contributor, Author

So turns out createResolutionCallbackFromFrame cannot be removed because it is used to create compiled methods from raw source (as string) as well. In that case, there is no static scope because we dont know where the string come from.

All reactions

```
1295  1295            _compile_and_register_class(obj, _rcb, qualified_name)
1296  1296            return obj
1297  1297        elif inspect.isfunction(obj) or inspect.ismethod(obj):
```

Terms Privacy Security Status Docs Contact Manage cookies
Do not share my personal information
© 2025 GitHub, Inc.